

GIF-Format



Kristijan Kreso 8241173
Mark Ian Braun 8175858

27. Februar 2026

Aktuelle Themen der Angewandten Informatik: Datenkompression

Johann Wolfgang Goethe-Universität Frankfurt am Main
FB Informatik und Mathematik
Institut für Informatik – Graphische Datenverarbeitung
Dozent: Dr.-Ing. The Anh Vuong

Erstellt am	02. Februar 2026
Status	fertig gestellt / Abgeben

Inhaltsverzeichnis

1	Thema	3
2	Theoretische Grundlagen	3
2.1	Geschichtlicher Hintergrund	3
2.2	Technische Merkmale	3
2.3	Aufbau einer GIF-Datei	4
3	Verfahren-Beschreibung	5
3.1	Prinzip und Initialisierung	5
3.2	Der Kompressionsprozess (Encoding)	6
3.3	Der Dekompressionsprozess (Decoding)	6
4	Anwendungsgebiet	7
4.1	Historische Entwicklung und Etablierung	7
4.2	Das Comeback in der Social-Media-Ära	7
4.3	Heutige Bedeutung	7
5	Qualitätsbewertung des Verfahrens	8
5.1	Best-Case: Maximale Redundanz	9
5.2	Schwächen und Worst-Case	9
6	Präsentation / Demonstration Kit	9
6.1	Zielsetzung und Konzept	9
6.2	Technische Umsetzung	10
6.3	Start der Applikation	10
6.4	Verifikation durch Dekomprimierung	16
6.4.1	Initialisierung der Dekodierung	16
6.4.2	Ablauf der Dekodierung und Bildrekonstruktion	17

1 Thema

Im Rahmen der folgenden Ausarbeitung werden das GIF-Format und das ihm zugrundeliegende Kompressionsverfahren analysiert. Der Fokus liegt dabei auf der **verlustfreien Datenkompression**, die durch den **Lempel-Ziv-Welch-Algorithmus (LZW)** realisiert wird. Im Anschluss wird ein webbasiertes Visualisierungstool vorgestellt, welches den Kompressions- und Dekompressionsprozess schrittweise demonstriert.

2 Theoretische Grundlagen

2.1 Geschichtlicher Hintergrund

Die Entwicklung des GIF-Formats („*Graphics Interchange Format*“) geht auf den Informatiker Steve Wilhite und sein Team beim US-amerikanischen Onlinedienst „CompuServe“ zurück. Die Veröffentlichung der ersten Spezifikation erfolgte im Juni 1987.

Primäres Ziel der Entwicklung war es, ein effizientes Farbformat zu schaffen, welches die bis dahin verbreiteten, technisch limitierten Schwarz-Weiß-Formate (wie z. B. XBM) ersetzte und gleichzeitig die Ladezeiten für Bilddateien über die damals langsamen Datenverbindungen minimierte [Adoa].

In der Folgezeit erschienen zwei Hauptversionen des Formats: die ursprüngliche Version „87a“ sowie die 1989 veröffentlichte Erweiterung „89a“, welche bis heute den Standard darstellt. Ein wesentlicher Unterschied dieser Versionen liegt in der Funktionsvielfalt: Während die erste Spezifikation primär auf *statische Bilder* ausgelegt war, führte der „89a“-Standard die Möglichkeit ein, mehrere Einzelbilder in einer Datei zu speichern. Durch diese Erweiterung wurde die Darstellung von *Animationen* realisierbar, was maßgeblich zur langfristigen Etablierung des Formats im Web beitrug.

2.2 Technische Merkmale

GIF zählt zu den *Bitmap-basierten* Grafikformaten. Dies bedeutet, dass den Bilddaten ein zweidimensionales Bildraster zugrunde liegt – eine Matrix aus Bildpunkten (Pixeln), wobei jeder Pixel eine spezifische Farbinformation trägt [Adob].

Ein charakteristisches Merkmal und zugleich die wesentliche Limitierung von GIF ist die Beschränkung der Farbtiefe auf maximal **8 Bit** (1 Byte) pro Pixel. Daraus ergibt sich, dass innerhalb eines einzelnen Bildframes höchstens $2^8 = 256$ verschiedene Farben gleichzeitig dargestellt werden können. Im Vergleich zu moderneren Formaten wie JPEG oder PNG, die eine Farbtiefe von bis zu 24 Bit (3 Byte) unterstützen und somit über 16 Millionen Farben ¹ abbilden können, ist die Farbdarstellung bei GIF deutlich reduziert.

Um die Datenmenge effizient zu verwalten, speichert GIF nicht für jeden Pixel den direkten RGB-Farbcode. Stattdessen basiert die technische Umsetzung auf einer *indizierten Farbpalette*. Hierbei werden die benötigten RGB-Werte in einer zentralen Tabelle (Color-Table) abgelegt, während für die Darstellung des eigentlichen Bildinhaltes lediglich die entsprechenden **Farbindizes** verwendet werden.

¹bei 24 Bit Farbtiefe (8 Bit pro RGB-Kanal) ergeben sich $2^{24} \approx 16,7$ Millionen mögliche Farbwerte.

Auf diese Weise wird im Gegensatz zum True-Color-Verfahren (3 Byte pro Pixel) nur 1 Byte pro Pixel benötigt. Jeder Farbindex referenziert damit eindeutig den jeweiligen Eintrag in der Farbpalette.

Tabelle 1: Schema der indizierten Farbpalette

Index	Farbwert
0	#00FF00
1	#DB1010
2	#00C8FF
3	#C100FF
...	...
255	#000000

Hinweis: Die gewählten Farben dienen lediglich als Beispiel.

Für die anschließende algorithmische Verarbeitung werden die Bilddaten nicht in ihrer zweidimensionalen Matrixform belassen. Stattdessen erfolgt eine Serialisierung der Daten: Das Bildraster wird zeilenweise, beginnend von der oberen linken Ecke bis zur unteren rechten Ecke, ausgelesen und in eine **lineare Sequenz** transformiert. Diese resultierende Kette aus Farbindizes dient als unmittelbarer Input für den Kompressionsalgorithmus.

Trotz quantitativer Einschränkungen bei der Farbtiefe arbeitet der eigentliche Kompressionsalgorithmus **verlustfrei**. Dies garantiert, dass die ursprüngliche Sequenz der Farbindizes nach der Dekompression „1:1“ rekonstruiert werden kann, wodurch die ursprünglichen Bilddaten ohne jeglichen Qualitätsverlust wiederhergestellt werden.

2.3 Aufbau einer GIF-Datei

Eine GIF-Datei ist modular aufgebaut und besteht aus verschiedenen Abschnitten [Com90].

- **Header** (Signatur und Version)
- **Logical Screen Descriptor** (Bilddimensionen)
- **Global Color Table** (Globale Farbtabelle)
- **Graphic Control Extension** (Steuerung von Transparenz und Animation)
- **Image Descriptor & Image Data** (Bildposition und komprimierte Daten)
- **Application Extension** (Zusatzfunktionen wie die Endlosschleife)
- **Trailer** (Datei-Abschluss)

Header: Der Header markiert den Beginn der Datei und besteht aus sechs Bytes. Die ersten drei Bytes enthalten die Signatur **GIF**, gefolgt von der Versionsnummer (meist 89a). Dieser Block dient der eindeutigen Identifikation des Dateityps durch den Decoder.

Logical Screen Descriptor: Dieser Abschnitt legt den „Rahmen“ des Bildes fest. Er definiert die Gesamtbreite und -höhe der Leinwand sowie den Hintergrundfarb-Index. Zudem signalisiert er über ein Flag, ob eine globale Farbtabelle folgt.

Global Color Table: Wie in dem vorherigen [Unterabschnitt](#) erläutert, fungiert dieser Block als zentrale „Mischpalette“. Er listet die bis zu 256 verwendeten RGB-Farbwerte auf, die referenziert werden können.

Graphic Control Extension (GCE): Dieser Block ist essenziell für moderne GIF-Funktionen. Er definiert die Verzögerungszeit bei Animationen sowie den Transparenz-Index, welcher bestimmt, welcher Farbwert der Palette als „durchsichtig“ behandelt wird.

Image Descriptor & Table Based Image Data: Der Descriptor gibt die exakten Koordinaten des Bildes auf der Leinwand an. Die darauf folgenden Bilddaten sind das Kernstück: Sie enthalten die mittels **LZW-Algorithmus** komprimierten Farbindizes (siehe Kapitel 3).

Application Extension: Hier werden anwendungsspezifische Informationen hinterlegt. Die bekannteste ist der *Netscape Looping Block*, der Webbrowsern mitteilt, wie oft eine Animationssequenz wiederholt werden soll.

Trailer: Das Ende der Datei wird durch ein einzelnes Byte mit dem Wert 0x3B (Hexadezimal für ein Semikolon) markiert. Es signalisiert dem Decoder den Abschluss des Datenstroms.

3 Verfahren-Beschreibung

Das funktionale Herzstück der GIF-Kompression ist der Lempel-Ziv-Welch-Algorithmus. Es handelt sich um ein verlustfreies, wörterbuchbasiertes Verfahren, dessen Effizienz darauf beruht, wiederkehrende Muster in der Datenfolge zu erkennen und durch kurze Referenzcodes zu ersetzen [\[Sal07\]](#).

3.1 Prinzip und Initialisierung

Die grundlegende Idee des LZW-Verfahrens im GIF-Format ist die Verwendung eines dynamischen Wörterbuchs. Damit der Prozess verlustfrei funktioniert, müssen Encoder und Decoder mit demselben Grundalphabet starten. Dieses Initialwörterbuch umfasst alle möglichen Einzelwerte der 8-Bit-Farbtiefe sowie zwei spezielle Steuerzeichen [\[Bla92\]](#).

Ein entscheidender Vorteil des LZW-Verfahrens ist, dass das während der Kompression aufgebaute Wörterbuch **nicht** in der Datei gespeichert werden muss. Da der Algorithmus einer festen Logik folgt, kann der Decoder das Wörterbuch während der Dekompression *on-the-fly* (schrittweise) absolut identisch rekonstruieren.

Tabelle 2: Initialisierung des LZW-Wörterbuchs

Sequenz	Code
0	0
1	1
...	...
255	255
CLEAR	256
EOI	257

Hinweis: Die Codes 256 (Clear Code) und 257 (End of Information) sind reservierte Steuerzeichen zur Steuerung des Decoders.

3.2 Der Kompressionsprozess (Encoding)

Der Encoder liest die lineare Sequenz der Farbindizes ein und versucht, immer längere Ketten im Wörterbuch zu finden. Der folgende Pseudocode beschreibt diesen Prozess:

```

LW := "" (Leeres Wort)
Solange noch Zeichen vorhanden:
    Z := lies nächstes Zeichen
    Falls (LW + Z) im Wörterbuch:
        LW := LW + Z
    Sonst:
        Füge (LW + Z) ins Wörterbuch ein
        Gib Code(LW) aus
        LW := Z
Falls LW != "":
    Gib Code(LW) aus
    
```

3.3 Der Dekompressionsprozess (Decoding)

Der Decoder empfängt die Codes als Input und schlägt sie in seinem Wörterbuch nach. Er ist asymmetrisch zum Encoder aufgebaut, da er auch einen Spezialfall behandeln muss, bei dem ein Code empfangen wird, der gerade erst im Wörterbuch erstellt wird.

```

c := Lies ersten Code aus dem Input-Stream
J := W[c] (Wörterbucheintrag für c)
Gib J aus
I := J
Solange noch Codes vorhanden:
    c := Lies nächsten Code
    Falls c im Wörterbuch:
    
```

```

J := W[c]
Sonst:
    J := I + I[0] (Spezialfall: Unbekannter Code)
    Füge (I + J[0]) ins Wörterbuch ein
    Gib J aus
I := J

```

4 Anwendungsgebiet

Die Relevanz des GIF-Formats hat sich über die Jahrzehnte stark gewandelt. Während es ursprünglich als technologische Notwendigkeit entstand, nimmt es heute eine Sonderrolle in der digitalen Kommunikation ein.

4.1 Historische Entwicklung und Etablierung

In den ersten Jahren des World Wide Web war GIF, neben dem Schwarz-Weiß-Bildformat XBM, das maßgebliche Standardformat für Grafiken. Ein wesentlicher Vorteil war dabei die Unterstützung von Farbe sowie – mit der Version 89a – die Möglichkeit, Transparenz und Animationen darzustellen.

Im Laufe der Jahre wurde GIF jedoch zunehmend von moderneren Grafikformaten überholt und in vielen Bereichen verdrängt. Grafikformate wie JPEG oder PNG ermöglichen im Vergleich zu GIF eine deutlich effizientere Kompression sowie eine weitaus höhere Farbtiefe. Durch diese technische Überlegenheit konnten Bildinhalte wesentlich detailgetreuer und speicherschonender dargestellt werden, wodurch GIF gerade für digitale Fotografien zunehmend an Bedeutung verlor [Adloa].

4.2 Das Comeback in der Social-Media-Ära

Trotz der technischen Limitierungen erlebten GIF-Animationen ab den 2010er Jahren ein überraschendes Comeback im Web. Als Hauptgrund hierfür gilt die Popularität von Bilder-Foren und Plattformen wie *Tumblr*, *Reddit* oder *4chan*. Dort wurde das Format genutzt, um kurze, geloopte Sequenzen wie Cinemagramme oder Stereogramme zu erstellen [Red13].

Ein entscheidender Meilenstein war das Jahr 2015, als auch Facebook begann, das Format nativ zu unterstützen, nachdem dies zuvor abgelehnt worden war. In der Folge integrierten nahezu alle großen Messenger-Dienste wie WhatsApp, Instagram und Slack eigene GIF-Suchmaschinen.

4.3 Heutige Bedeutung

Zusammenfassend lässt sich festhalten, dass das GIF-Format trotz seiner technologischen Unterlegenheit gegenüber moderneren Bildformaten in vielen Bereichen des heutigen Internets eine unverzichtbare Sonderrolle behält. Dies hängt vor allem mit der Option der

Animationsdarstellung und der universellen Kompatibilität zusammen. Heute wird das Format primär in folgenden Bereichen eingesetzt:

- **Digitale Kommunikation (Memes und Loops):** GIF fungiert als globales kulturelles Ausdrucksmittel für kurze, tonlose Reaktionen und Animationen, die in sozialen Netzwerken und Messengern fest integriert sind.
- **Grafikelemente (Logos und Icons):** Für einfache Grafiken mit geringer Farbanzahl bietet das Format eine verlustfreie Darstellung, die auf allen Endgeräten ohne Qualitätsverlust gerendert werden kann.
- **E-Mail-Marketing:** Da viele E-Mail-Clients die Einbettung von Videodateien blockieren, bleibt das GIF-Format die einzige verlässliche Methode, um bewegte Inhalte direkt in Newslettern oder Werbe-Mails darzustellen.

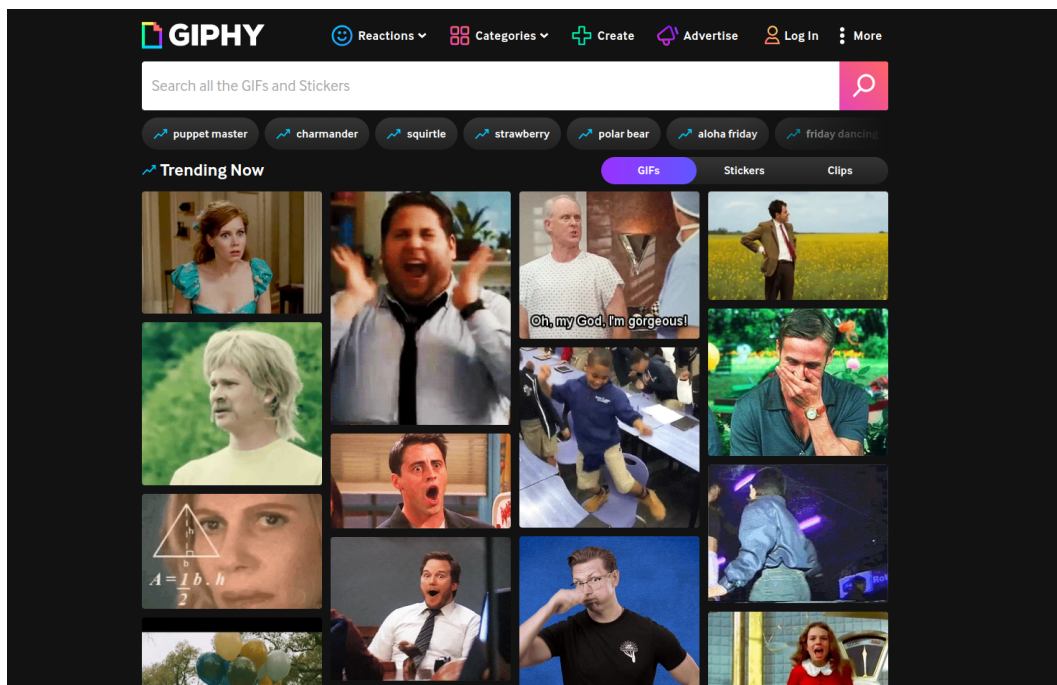


Abbildung 1: Die Plattform GIPHY veranschaulicht die heutige Rolle des GIF-Formats als zentrales Medium für die visuelle Kommunikation und als Datenbank für animierte Kurzsequenzen.

5 Qualitätsbewertung des Verfahrens

Die Kompressionsgüte des im GIF-Format verwendeten LZW-Algorithmus ist maßgeblich von der Struktur bzw. der Redundanz der zugrundeliegenden Bilddaten abhängig.

Merksatz 5.1 (Effizienz von LZW). Der LZW-Algorithmus arbeitet am effektivsten bei Daten mit vielen sich wiederholenden Sequenzmustern. Je höher die **Redundanz** innerhalb der Input-Sequenz, desto höher ist die resultierende Kompressionsrate.

5.1 Best-Case: Maximale Redundanz

Der Idealfall für das Verfahren tritt bei großen, einfarbigen Flächen auf, wie sie häufig in Logos, Icons oder technischen Tabellen vorkommen.

In diesem Szenario entstehen extrem lange Ketten identischer Farbindizes. Da das Wörterbuch während des Encoding-Prozesses schrittweise immer längere Sequenzen speichert, können riesige Bildbereiche durch einen einzigen, relativ kurzen Referenzcode dargestellt werden. Die Kompression ist hier maximal effizient, da die Menge der ausgegebenen Codes im Vergleich zur Anzahl der ursprünglichen Pixel minimal ist.

5.2 Schwächen und Worst-Case

Die Kompressionsleistung sinkt signifikant, sobald das Bild eine hohe Entropie (Unordnung) aufweist. Dies ist insbesondere bei folgenden Bildinhalten der Fall:

- Komplexe Farbverläufe und detailreiche Fotografien.
- Bildrauschen oder stark texturierte Oberflächen.
- Eine hohe Anzahl unterschiedlicher, unregelmäßig wechselnder Farben.

Definition 5.1 (Worst-Case der Kompression). Der theoretische **Worst-Case** tritt ein, wenn alle aufeinanderfolgenden Pixel eines Bildes unterschiedliche Farben aufweisen.

Beispiel: Betrachten wir ein Raster von 5×5 Pixeln, bei dem jeder Pixel eine eindeutige Farbe trägt (z. B. die Indizes $1, 2, 3, \dots, 25$). Da keine Sequenz doppelt auftritt, kann der Algorithmus keine Muster in das Wörterbuch übernehmen, die später erneut referenziert werden könnten. Der Output wäre gleich wie die ursprüngliche Sequenz. Die effektive Kompressionsrate wäre in diesem Fall gleich Null.

6 Präsentation / Demonstration Kit

Das im Rahmen dieser Arbeit entwickelte webbasierte Visualisierungstool, der **GIF Compression Visualizer**, dient der interaktiven Vermittlung des LZW-Algorithmus. Das primäre Ziel der Anwendung ist es, die Abläufe der Wörterbuchbildung und Code-Generierung schrittweise und visuell nachvollziehbar zu machen.

6.1 Zielsetzung und Konzept

Die Anwendung wurde entworfen, um die theoretischen Konzepte aus [Kapitel 3](#) in eine praktische Lernumgebung zu überführen. Anstatt den Kompressionsvorgang als geschlossenen Prozess zu behandeln, ermöglicht das Tool eine detaillierte Beobachtung der

Datenverarbeitung in Echtzeit. Ein wesentlicher Aspekt ist dabei die Flexibilität: Nutzer können eigene Bilddateien hochladen, wodurch die Auswirkungen unterschiedlicher Bildstrukturen auf die Kompressionseffizienz (wie in [Kapitel 5](#) analysiert) direkt sichtbar werden.

6.2 Technische Umsetzung

Das Tool wurde als moderne Web-Anwendung realisiert. Wir haben dafür **HTML5**, **CSS** und **JavaScript** verwendet, da dies eine plattformunabhängige Nutzung ohne zusätzliche Installation ermöglicht. Die gesamte Logik des LZW-Algorithmus wurde in JavaScript implementiert, um eine flüssige, clientseitige Visualisierung der Rechenschritte zu garantieren.

6.3 Start der Applikation

Die Anwendung wird durch das Öffnen der `index.html` über einen lokalen Webserver (z. B. auf einem definierten Localhost-Port) im Browser gestartet. Um die visuelle Darstellung der Benutzeroberfläche, insbesondere die Kontraste der Syntax-Hervorhebung, optimal zu nutzen, sollte der Browser im **Dark Mode** betrieben werden.

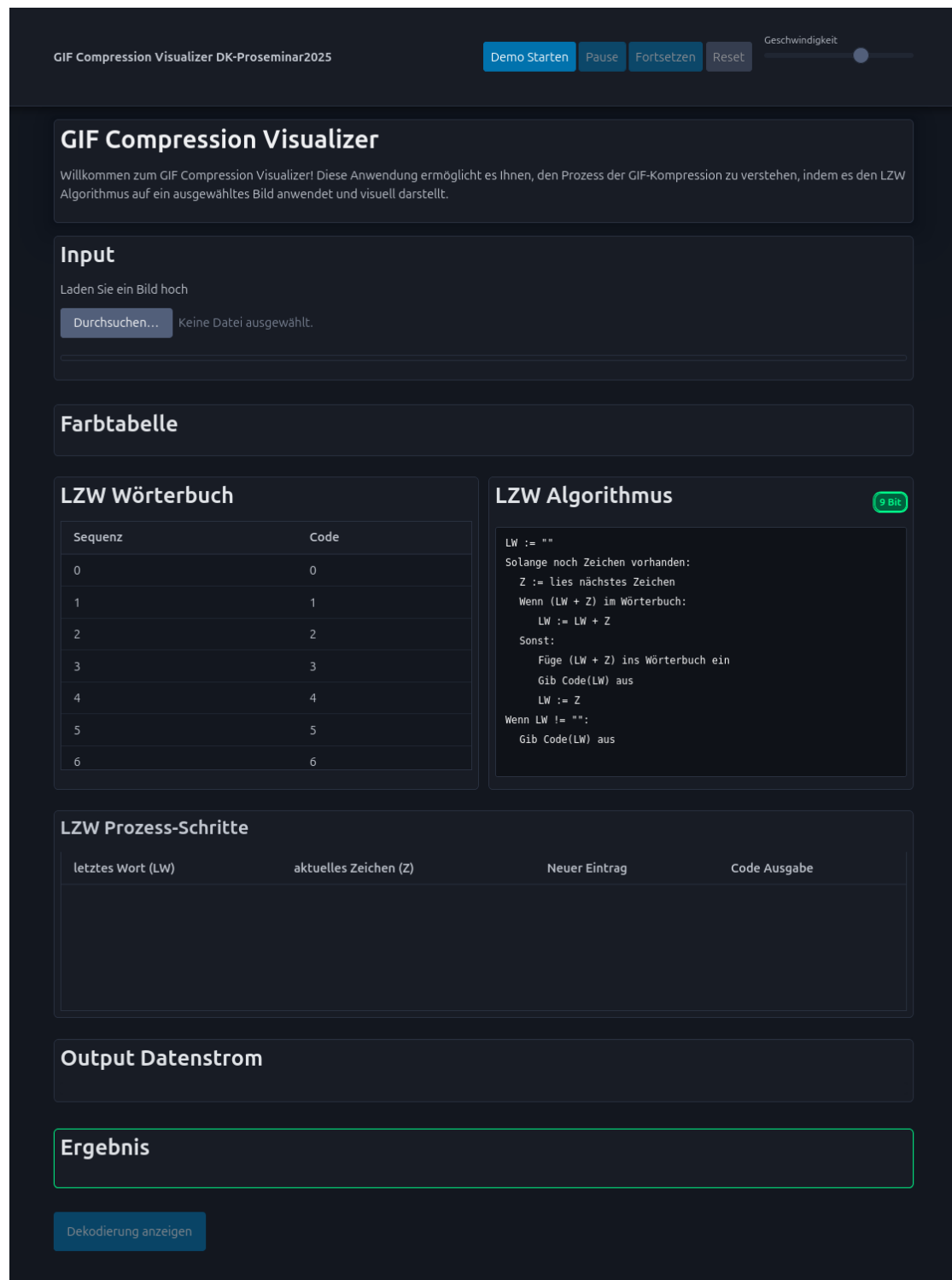


Abbildung 2: Die Initialansicht des GIF Compression Visualizers vor dem Daten-Upload.

Nach der Initialisierung präsentiert sich das Tool mit einer leeren Arbeitsfläche, wie in [Abbildung 2](#) zu sehen ist. Die Benutzeroberfläche ist intuitiv gestaltet:

- **Header-Sektion:** Enthält die Titelzeile sowie die zentralen Steuerungselemente (*Demo Starten*, *Pause*, *Fortsetzen*, *Reset*) und den Geschwindigkeitsregler für die Schritt-für-Schritt-Visualisierung.
- **Input-Modul:** Dies ist der erste Interaktionspunkt für den Nutzer. Über die Schaltfläche „Durchsuchen...“ kann eine lokale Bilddatei ausgewählt und in den Visualisierer geladen werden.
- **Vorschau-Bereich:** Sobald ein Bild ausgewählt wurde, dient die Sektion unter dem Upload-Feld als visuelle Referenz für das zu komprimierende Ausgangsmaterial.

Nachdem die Anwendung erfolgreich gestartet wurde, erfolgt die aktive Interaktion des Nutzers mit dem System.

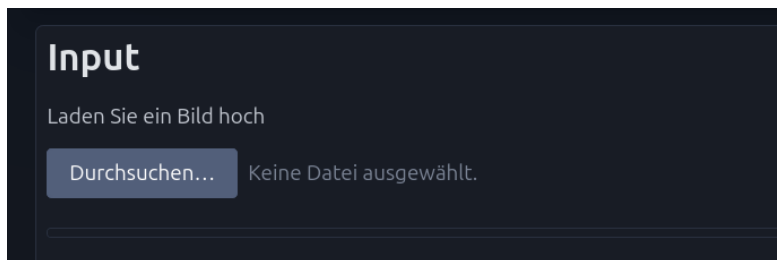


Abbildung 3: Detailansicht des Input-Moduls mit der Upload-Schaltfläche.

Wie in [Abbildung 3](#) dargestellt, kann der Anwender nun auf die Schaltfläche „Durchsuchen...“ klicken. Es öffnet sich der Dateidialog des Betriebssystems, welcher es erlaubt, ein beliebiges Bild auszuwählen. In diesem Moment liest das Tool die Rohdaten der Grafik ein und beginnt im Hintergrund mit der Vorbereitung der Datenstrukturen für den Kompressionsvorgang.

Sobald eine Datei ausgewählt wurde (in diesem Beispiel die Datei `test_very_small.gif`), visualisiert das Tool die Transformation der Bilddaten in eine verarbeitbare Form.

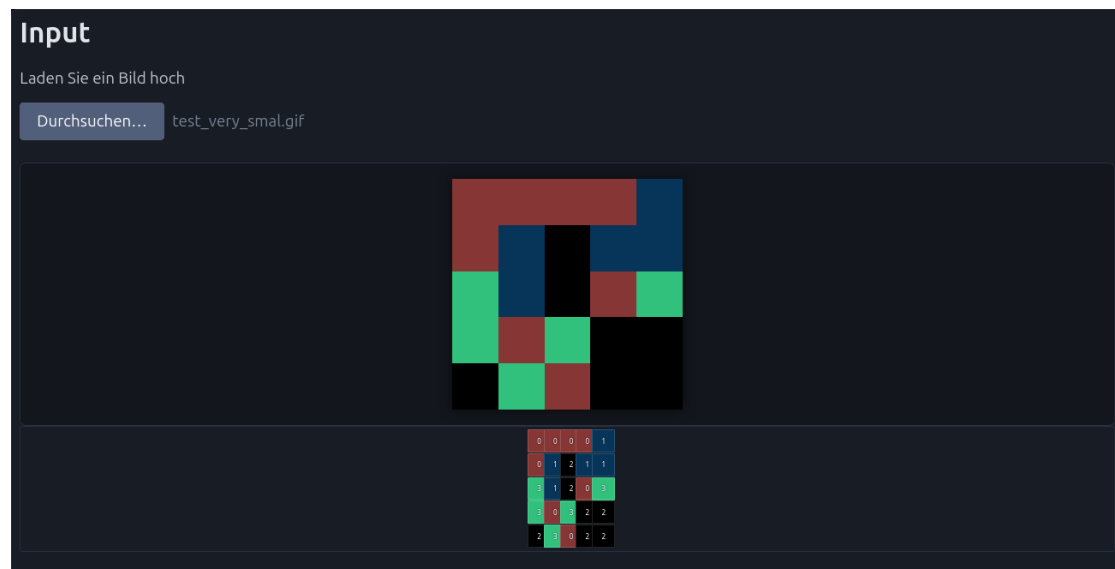


Abbildung 4: Darstellung des geladenen Testbildes und der daraus extrahierten Farbindizes.

Wie in [Abbildung 4](#) zu sehen ist, wird das hochgeladene Bild im zentralen Vorschaubereich angezeigt. Direkt darunter generiert das Tool ein korrespondierendes Raster, welches die einzelnen Pixel durch ihre jeweiligen Werte aus der Farbtabelle (Farbindizes) ersetzt.

Diese Matrix dient als direkte **Input-Sequenz** für den LZW-Algorithmus. Durch diese Gegenüberstellung wird für den Anwender sofort ersichtlich, wie das eigentliche Bild durch entsprechende Farbindizes repräsentiert wird.

Nachdem die Bilddaten erfolgreich eingelesen und als Index-Raster visualisiert wurden, kann die eigentliche Kompressions-Simulation initiiert werden.

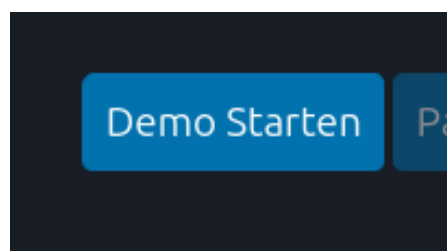


Abbildung 5: Die Schaltfläche zum Starten der interaktiven Simulation.

Durch Betätigen der Schaltfläche „Demo Starten“ (siehe [Abbildung 5](#)) beginnt das Tool mit der schrittweisen Abarbeitung der Input-Sequenz. In diesem Modus durchläuft der Algorithmus das Pixel-Raster von oben links nach unten rechts.

Der Anwender kann nun beobachten, wie das Tool die Steuerung übernimmt: Die

aktuell verarbeiteten Pixel im Raster werden hervorgehoben, während simultan die entsprechenden Zeilen im Pseudocode-Fenster aufleuchten und das Wörterbuch dynamisch erweitert wird. Über den Geschwindigkeitsregler kann dieser Prozess jederzeit verlangsamt werden, um die Entstehung einzelner Code-Ausgaben im Detail nachzuvollziehen.

Während die Simulation läuft, dokumentiert das Tool die interne Logik des Algorithmus durch eine synchrone Aktualisierung aller Module.

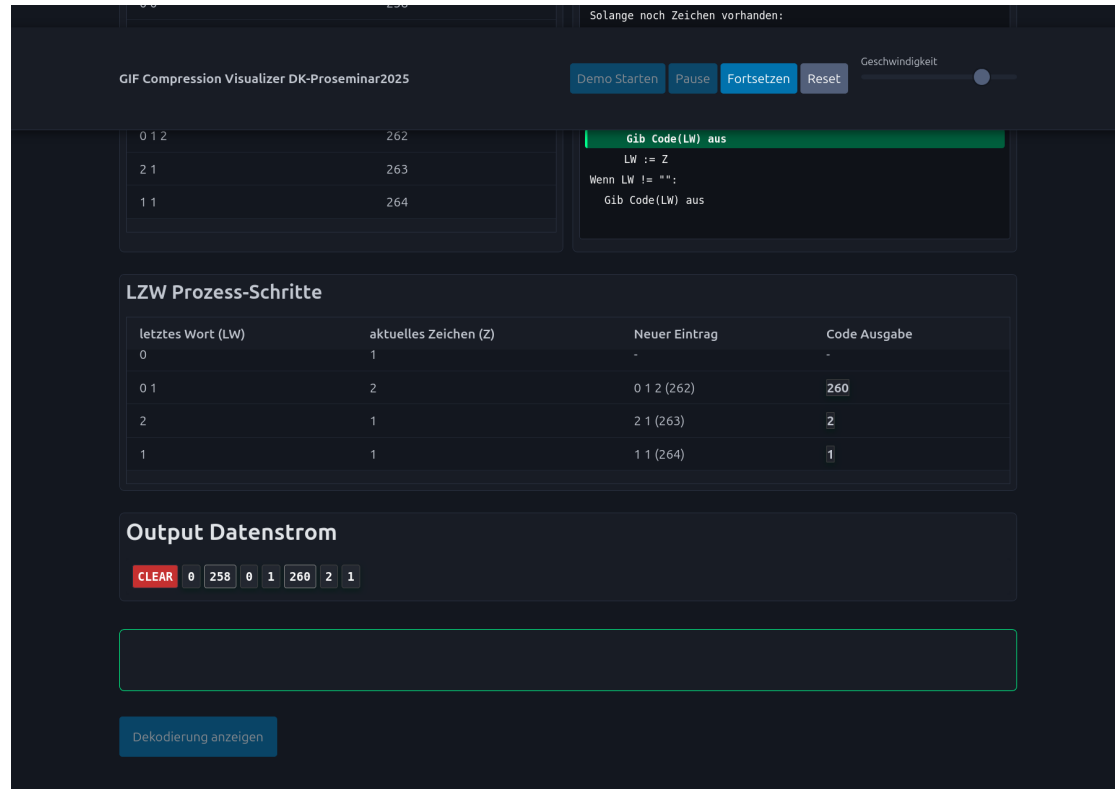


Abbildung 6: Live-Visualisierung der LZW-Prozessschritte, des Pseudocode-Status und des Output-Datenstroms.

In [Abbildung 6](#) wird die operative Arbeitsweise des Visualisierers deutlich:

- **Pseudocode-Tracking:** Im Fenster *LZW Algorithmus* wird die aktuell ausgeführte Zeile (hier: `Gib Code(LW) aus`) farblich hervorgehoben. Dies ermöglicht es dem Nutzer, die abstrakte Programmlogik direkt mit der Datenmanipulation zu verknüpfen.
- **Prozess-Tabelle:** Das Modul *LZW Prozess-Schritte* liefert eine lückenlose Historie. Es zeigt präzise, wie aus der Kombination von *letztem Wort* (LW) und *aktuellem Zeichen* (Z) neue Einträge für das Wörterbuch generiert werden (z. B. Sequenz 0 1 2 mit dem neuen Code 262).

- **Output Datenstrom:** Hier wird die resultierende Codefolge in Echtzeit aufgebaut. Besonders hervorzuheben ist die automatische Voranstellung des *Clear Codes* (Index 256), der den Beginn eines GIF-Datenstroms markiert.

Der Nutzer kann durch diese parallele Darstellung genau nachvollziehen, warum bestimmte Codes ausgegeben werden und wie das Wörterbuch sukzessiv mit Mustern gefüllt wird, die bei erneutem Auftreten die Datenmenge reduzieren.

Nachdem der Algorithmus die gesamte Input-Sequenz verarbeitet hat, wird der Prozess mit einer detaillierten statistischen Auswertung abgeschlossen.

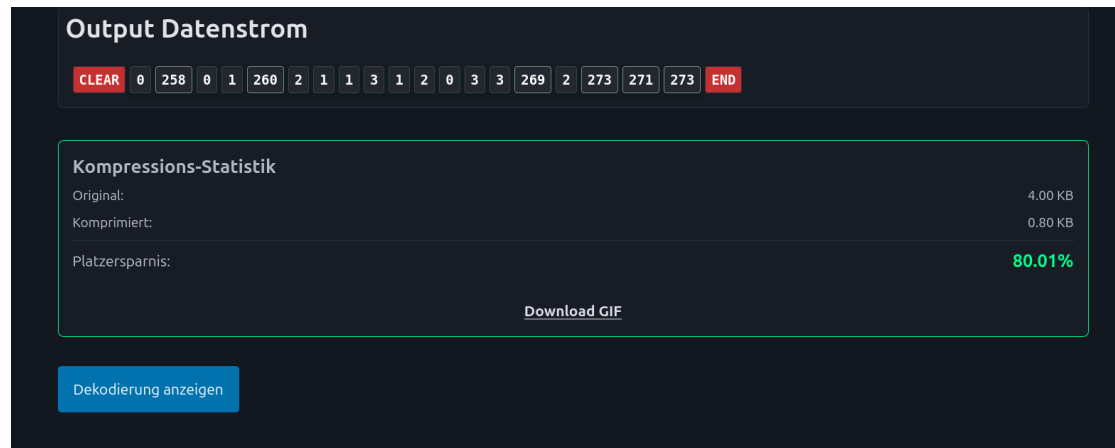


Abbildung 7: Abschließende Kompressions-Statistik und Download-Option.

Wie in [Abbildung 7](#) illustriert, blendet das Tool nach Beendigung des Vorgangs das Modul *Kompressions-Statistik* ein. Hier werden die Ergebnisse quantifiziert:

- **Originalgröße:** Die Dateigröße des unverarbeiteten Ausgangsbildes (hier: 4.00 KB).
- **Komprimierte Größe:** Die resultierende Größe nach der LZW-Kodierung (hier: 0.80 KB).
- **Platzersparnis:** Die prozentuale Reduktion der Datenmenge im Verhältnis zur Originaldatei. In diesem Beispiel wurde eine beachtliche Effizienz von **80.01%** erzielt.

Zusätzlich bietet das Tool die Möglichkeit, die generierte Datei über die Schaltfläche „Download GIF“ lokal zu speichern. Dieser finale Schritt untermauert die Praxistauglichkeit der Anwendung: Der Nutzer hat nicht nur einen theoretischen Prozess beobachtet, sondern tatsächlich eine Grafikdatei erstellt.

6.4 Verifikation durch Dekomprimierung

Ein wesentliches Qualitätsmerkmal des LZW-Verfahrens ist die vollständige Reversibilität der Datenkompression. Um diesen Aspekt zu überprüfen, bietet die Anwendung nach Abschluss der Kodierung eine Funktion zur Dekomprimierung an.

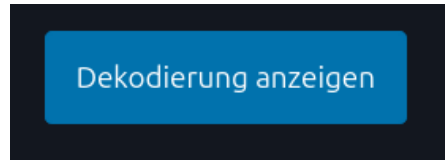


Abbildung 8: Schaltfläche zur Einleitung des Dekomprimierungsvorgangs.

Durch Betätigen der Schaltfläche „Dekodierung anzeigen“ (siehe [Abbildung 8](#)) ist es möglich den inversen Prozess zu beobachten. Dabei wird der zuvor generierte Output-Datenstrom wieder in ein Pixel-Raster übersetzt.

Dieser Schritt dient der abschließenden Kontrolle: Der Anwender kann das rekonstruierte Bild direkt mit dem ursprünglichen Input-Bild vergleichen. Die identische Darstellung beider Raster bestätigt die theoretischen Ausführungen zur Verlustfreiheit des GIF-Formats und schließt die praktische Demonstration des Verfahrens ab.

6.4.1 Initialisierung der Dekodierung

Mit Betätigen der Schaltfläche wechselt die Ansicht in das Dekodierungs-Modul. Hier wird ein wesentlicher Vorteil des LZW-Verfahrens sichtbar: Die Information über die dynamisch erstellten Wörterbucheinträge muss nicht in der Datei gespeichert sein, da der Decoder diese während des Lesevorgangs aus dem Datenstrom rekonstruiert.

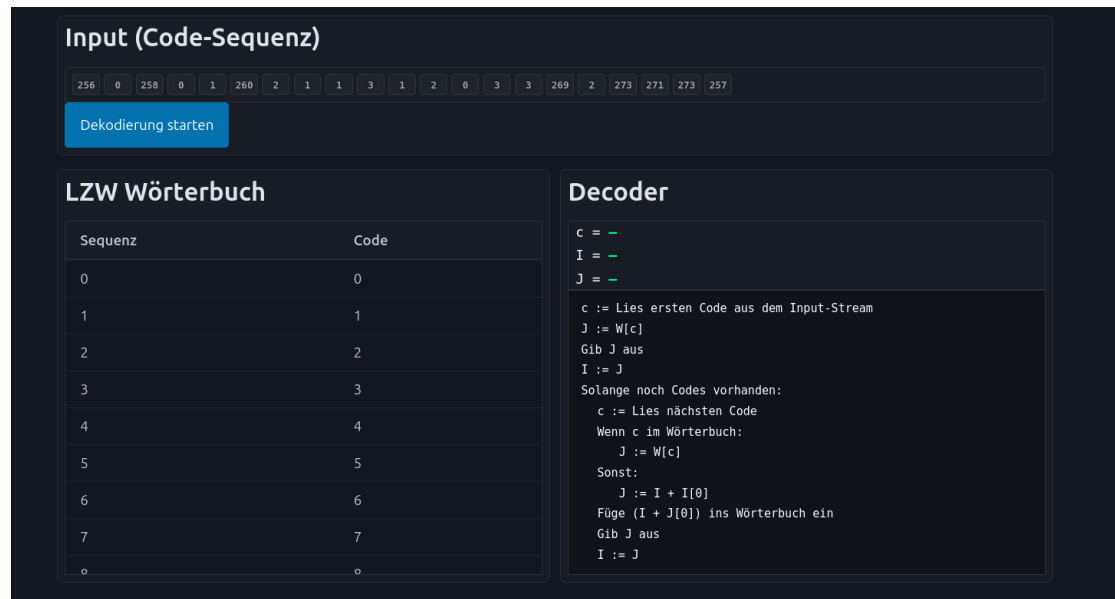


Abbildung 9: Startansicht des Decoders mit initialisiertem Basis-Wörterbuch.

In der Startansicht des Decoders (siehe [Abbildung 9](#)) ist zu erkennen, dass lediglich das Basis-Wörterbuch (die Farbindizes) vorbesetzt ist. Die zuvor generierte Code-Folge wird nun als *Input (Code-Sequenz)* bereitgestellt. Um den Prozess zu starten, steht dem Anwender unterhalb der Sequenz die Schaltfläche „Dekodierung starten“ zur Verfügung. Analog zum Encoder-Modul wird auch hier die Logik durch einen spezifischen Pseudocode sowie eine Status-Anzeige der Variablen C , I und J visualisiert.

6.4.2 Ablauf der Dekodierung und Bildrekonstruktion

Während der Simulation der Dekodierung lässt sich beobachten, wie der Algorithmus die Code-Sequenz liest und daraus die ursprünglichen Farbmuster ableitet. Dieser Prozess wird durch die synchrone Aktualisierung des Decoder-Logikfensters und der Bildausgabe visualisiert.

GIF Compression Visualizer DK-Proseminar2025
Demo Starten
Pause
Fortsetzen
Reset
Geschwindigkeit:

LZW Wörterbuch

Sequenz	Code
253	253
254	254
255	255
CLEAR	256
END	257
0 0	258
0 0 0	259
0 1	260

Decoder

```

c = 260
I = 1
J = -

c := Lies ersten Code aus dem Input-Stream
J := W[c]
Gib J aus
I := J

Solange noch Codes vorhanden:
  c := Lies nächsten Code
  Wenn c im Wörterbuch:
    J := W[c]
  Sonst:
    J := I + I[0]
  Füge (I + J[0]) ins Wörterbuch ein
  Gib J aus
I := J

```

Abbildung 10: Visualisierung des Dekodierungsvorgangs mit aktiver Zeilenhervorhebung im Pseudocode und Variablenstatus.

In [Abbildung 10](#) wird der aktuelle Zustand der Dekodierung dargestellt. Das Tool hebt die aktive Logik-Zeile im Pseudocode hervor (hier die Prüfung, ob ein Code bereits im Wörterbuch existiert). Gleichzeitig werden die Arbeitsvariablen c (aktueller Code), I (vorherige Sequenz) und J (neu ausgegebene Sequenz) in Echtzeit aktualisiert. Parallel dazu wächst das *LZW Wörterbuch* wieder an, indem die neu gelernten Sequenzen (z. B. Index 258 für die Folge 0 0) eingetragen werden.

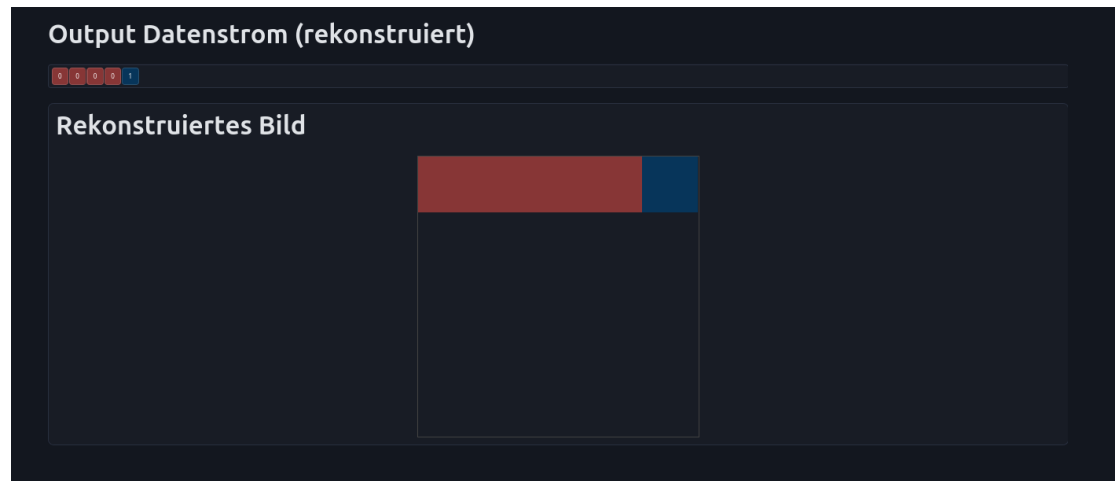


Abbildung 11: Schrittweise Rekonstruktion der Farbindizes und des grafischen Bildes.

Simultan zur algorithmischen Verarbeitung erfolgt die visuelle Ausgabe im Modul *Rekonstruiertes Bild* (siehe [Abbildung 11](#)). Der *Output Datenstrom (rekonstruiert)* zeigt die wiedergewonnenen Farbindizes, während im darunterliegenden Feld die entsprechenden Pixel in der korrekten Farbgebung gezeichnet werden.

Durch diese schrittweise Rekonstruktion wird dem Anwender verdeutlicht, dass am Ende des Prozesses eine bitgenaue Kopie des ursprünglichen Bildes entsteht. Die Demonstration endet mit der vollständigen Wiederherstellung der Grafik, was die Korrektheit und Verlustfreiheit der implementierten LZW-Logik bestätigt.



Abbildung 12: Vollständig rekonstruiertes Bild nach Abschluss der Dekodierung.

Wie in [Abbildung 12](#) ersichtlich ist, führt die Abarbeitung der gesamten Code-Sequenz zur vollständigen Wiederherstellung der ursprünglichen Grafik. Das rekonstruierte Pixel-Raster entspricht in jedem Farbwert exakt dem in [Abbildung 4](#) gezeigten Input-Raster.

Dieser direkte Vergleich verifiziert die **Verlustfreiheit** des LZW-Algorithmus: Trotz der erheblichen Datenkompression (in diesem Fall ca. 80 %, siehe [Abbildung 7](#)) gehen keine Bildinformationen verloren. Das Demonstration Kit bestätigt somit praktisch, dass das im GIF-Standard verwendete Verfahren eine effiziente Archivierung ohne qualitative Einbußen ermöglicht. Damit ist die Funktionskette der Anwendung – von der Analyse über die Kompression bis hin zur fehlerfreien Rekonstruktion – erfolgreich abgeschlossen.

Literatur

- [Adoa] Adobe Inc. *GIF-Datei: Eigenschaften des Graphics Interchange Formats*. URL: <https://www.adobe.com/de/creativecloud/file-types/image/raster/gif-file.html> (besucht am 06.02.2026).
- [Adob] Adobe Inc. *Rasterdateien*. Grundlagen zu Bitmap-Formaten. URL: <https://www.adobe.com/de/creativecloud/file-types/image/raster.html> (besucht am 06.02.2026).
- [Bla92] Steve Blackstock. *LZW and GIF explained*. Carnegie Mellon University. 1992. URL: <https://www.cs.cmu.edu/~guyb/real-world/compress/lzexplained.html> (besucht am 06.02.2026).
- [Com90] CompuServe Incorporated. *Graphics Interchange Format (Version 89a)*. Technical Specification. CompuServe Incorporated. Columbus, Ohio, Juli 1990. URL: <https://www.w3.org/Graphics/GIF/spec-gif89a.txt> (besucht am 06.02.2026).
- [Red13] Redaktion derStandard.at. *Retrochic: Animierte GIFs feiern ihr Comeback*. Bedeutung von GIF in der digitalen Kommunikation. 2013. URL: <https://www.derstandard.at/story/1360681458046/animierte-gifs-feiern-ihr-comeback> (besucht am 25.02.2026).
- [Sal07] David Salomon. *Data Compression: The Complete Reference*. 4th. Abschnitt zum LZW-Algorithmus und GIF-Implementierung. London: Springer, 2007, S. 199–208, 225. ISBN: 978-1-84628-603-2.